



(12) **United States Patent**
Chintala et al.

(10) **Patent No.:** **US 12,386,823 B2**
(45) **Date of Patent:** ***Aug. 12, 2025**

(54) **MULTIPLE ENVIRONMENTS AND
FUNCTIONS IN A DATABASE SYSTEM**

(58) **Field of Classification Search**
CPC G06F 16/2448; G06F 16/27; G06F 16/248;
G06F 8/433

(71) Applicant: **Snowflake Inc.**, Bozeman, MT (US)

(Continued)

(72) Inventors: **Srilakshmi Chintala**, Seattle, WA (US);
Chong Han, Bellevue, WA (US);
Albert L. Hu, Seattle, WA (US); **Nitya
Kumar Sharma**, Bellevue, WA (US);
Igor Zinkovsky, Redmond, WA (US)

(56) **References Cited**
U.S. PATENT DOCUMENTS

11,385,889 B2 7/2022 Fender et al.
11,640,822 B2 5/2023 Lewis et al.

(Continued)

(73) Assignee: **Snowflake Inc.**, Bozeman, MT (US)

(*) Notice: Subject to any disclaimer, the term of this
patent is extended or adjusted under 35
U.S.C. 154(b) by 0 days.

This patent is subject to a terminal dis-
claimer.

OTHER PUBLICATIONS

“U.S. Appl. No. 18/051,483, Non Final Office Action mailed May
10, 2023”, 15 pgs.

(Continued)

Primary Examiner — Michelle N Owyang

(74) *Attorney, Agent, or Firm* — Schwegman Lundberg &
Woessner, P.A.

(21) Appl. No.: **18/429,286**

(22) Filed: **Jan. 31, 2024**

(65) **Prior Publication Data**
US 2024/0168947 A1 May 23, 2024

Related U.S. Application Data

(63) Continuation of application No. 18/051,483, filed on
Oct. 31, 2022, now Pat. No. 11,928,110.
(Continued)

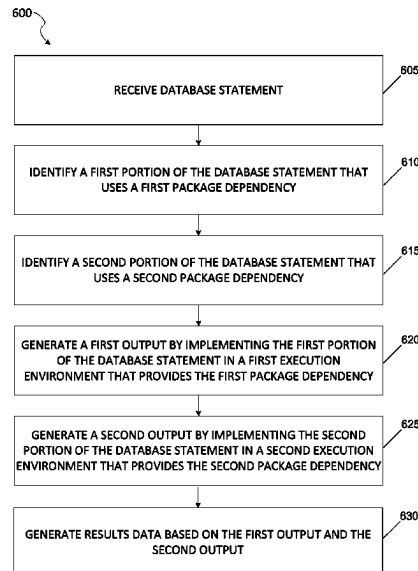
(51) **Int. Cl.**
G06F 16/00 (2019.01)
G06F 16/242 (2019.01)
(Continued)

(52) **U.S. Cl.**
CPC **G06F 16/2448** (2019.01); **G06F 16/248**
(2019.01); **G06F 16/27** (2019.01); **G06F 8/433**
(2013.01)

(57) **ABSTRACT**

Multiple execution environments are established for pro-
cessing database statements with diverse package dependen-
cies. A database system receives a user application and
identifies, via hardware processors, multiple package depen-
dencies required by the application, including at least a first
and a second package dependency. Upon receiving a data-
base statement from a user that invokes these dependencies,
the system retrieves the necessary packages from a reposi-
tory. It then creates a first execution environment incorpo-
rating the first package dependency and a second execution
environment with the second package dependency. Utilizing
the multiple execution environments, the system generates
results data in response to the database statement, effectively
managing and executing user-defined functions that rely on
different versions or types of package dependencies within
a unified database framework.

24 Claims, 7 Drawing Sheets



Related U.S. Application Data

(60) Provisional application No. 63/268,583, filed on Feb. 25, 2022.

(51) **Int. Cl.**

G06F 16/248 (2019.01)

G06F 16/27 (2019.01)

G06F 8/41 (2018.01)

(58) **Field of Classification Search**

USPC 707/600–899

See application file for complete search history.

(56) **References Cited**

U.S. PATENT DOCUMENTS

2011/0035724 A1 2/2011 Kettley et al.
 2015/0199188 A1* 7/2015 Mantripragada G06F 21/121
 717/101
 2020/0057633 A1* 2/2020 Topiwala G06F 8/433
 2021/0048998 A1 2/2021 Myers et al.
 2021/0124739 A1 4/2021 Karanasos et al.

2021/0141645 A1 5/2021 Kramer et al.
 2021/0311859 A1 10/2021 De Sousa Bispo et al.
 2022/0171738 A1 6/2022 Lee et al.
 2022/0179627 A1* 6/2022 Bagadia G06F 8/433
 2023/0125342 A1 4/2023 Krishna et al.
 2023/0153092 A1* 5/2023 Torkelson G06F 8/71
 717/175
 2023/0273918 A1 8/2023 Chintala et al.

OTHER PUBLICATIONS

“U.S. Appl. No. 18/051,483, Response filed Aug. 10, 2023 to Non Final Office Action mailed May 10, 2023”, 12 pgs.

“U.S. Appl. No. 18/051,483, Examiner Interview Summary mailed Aug. 15, 2023”, 2 pgs.

“U.S. Appl. No. 18/051,483, Final Office Action mailed Aug. 18, 2023”, 18 pgs.

“U.S. Appl. No. 18/051,483, Response filed Nov. 20, 2023 to Final Office Action mailed Aug. 18, 2023”, 17 pgs.

“U.S. Appl. No. 18/051,483, Notice of Allowance mailed Jan. 23, 2024”, 8 pgs.

* cited by examiner

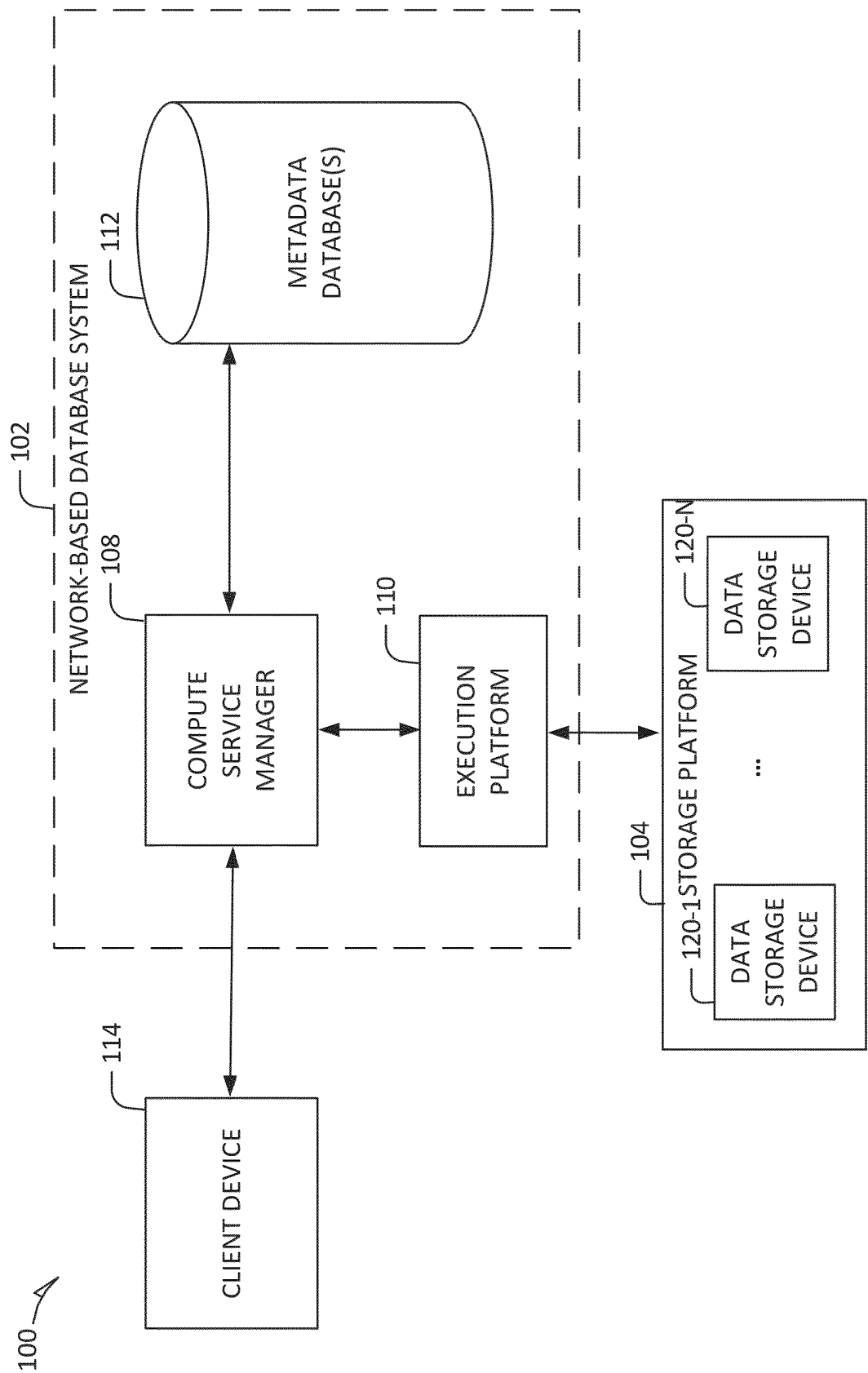


FIG. 1

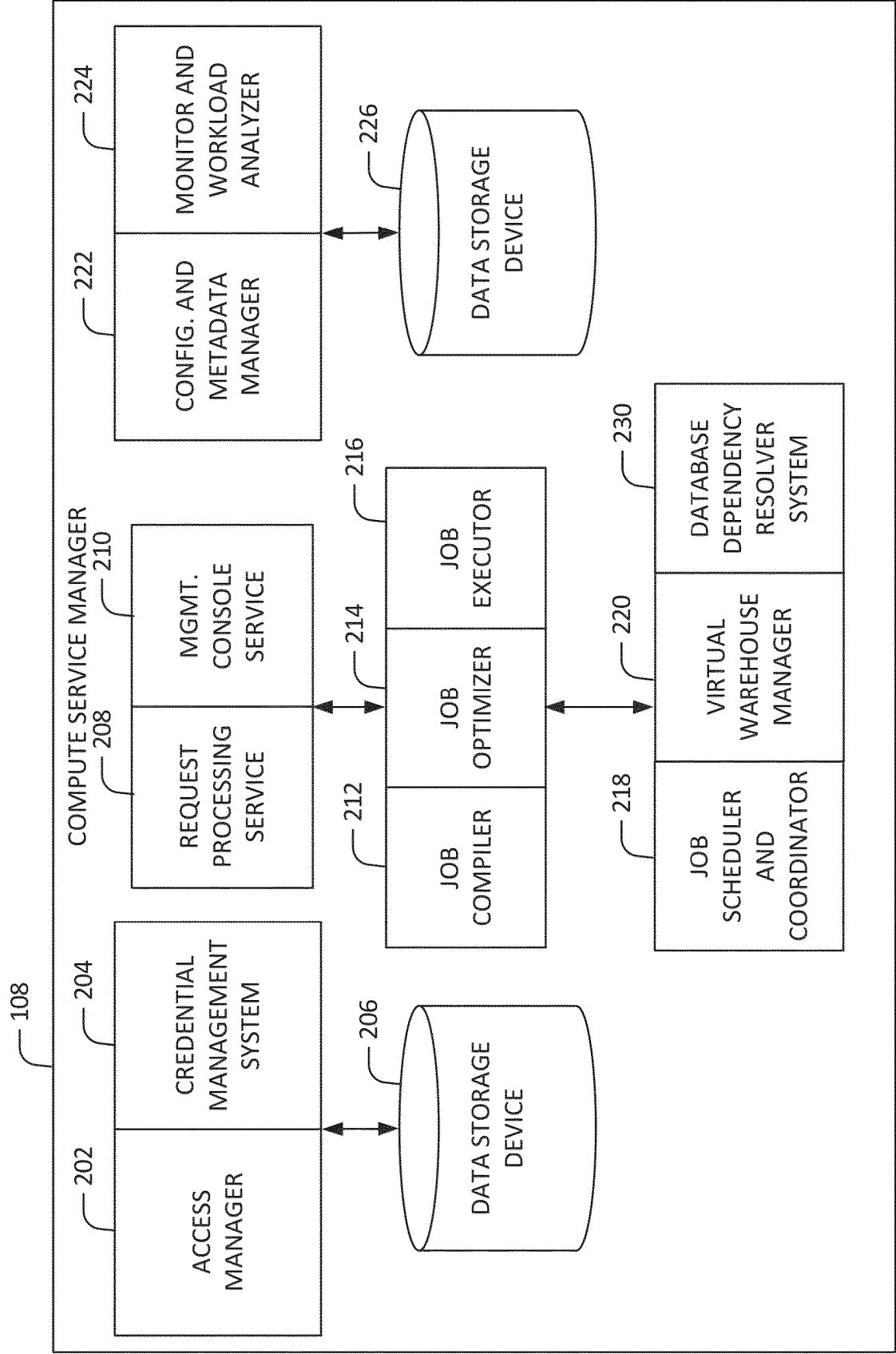


FIG. 2

110 

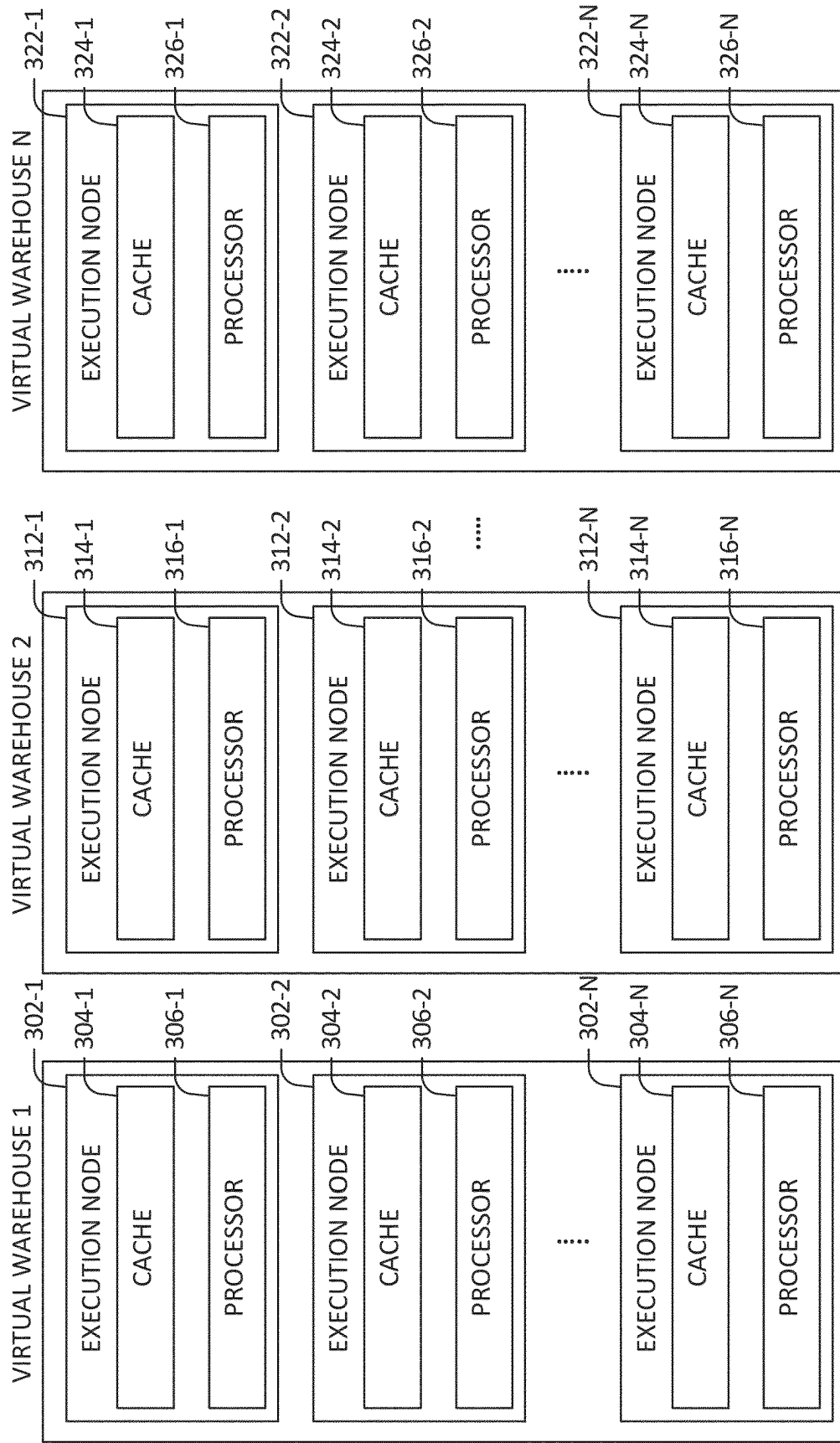


FIG. 3

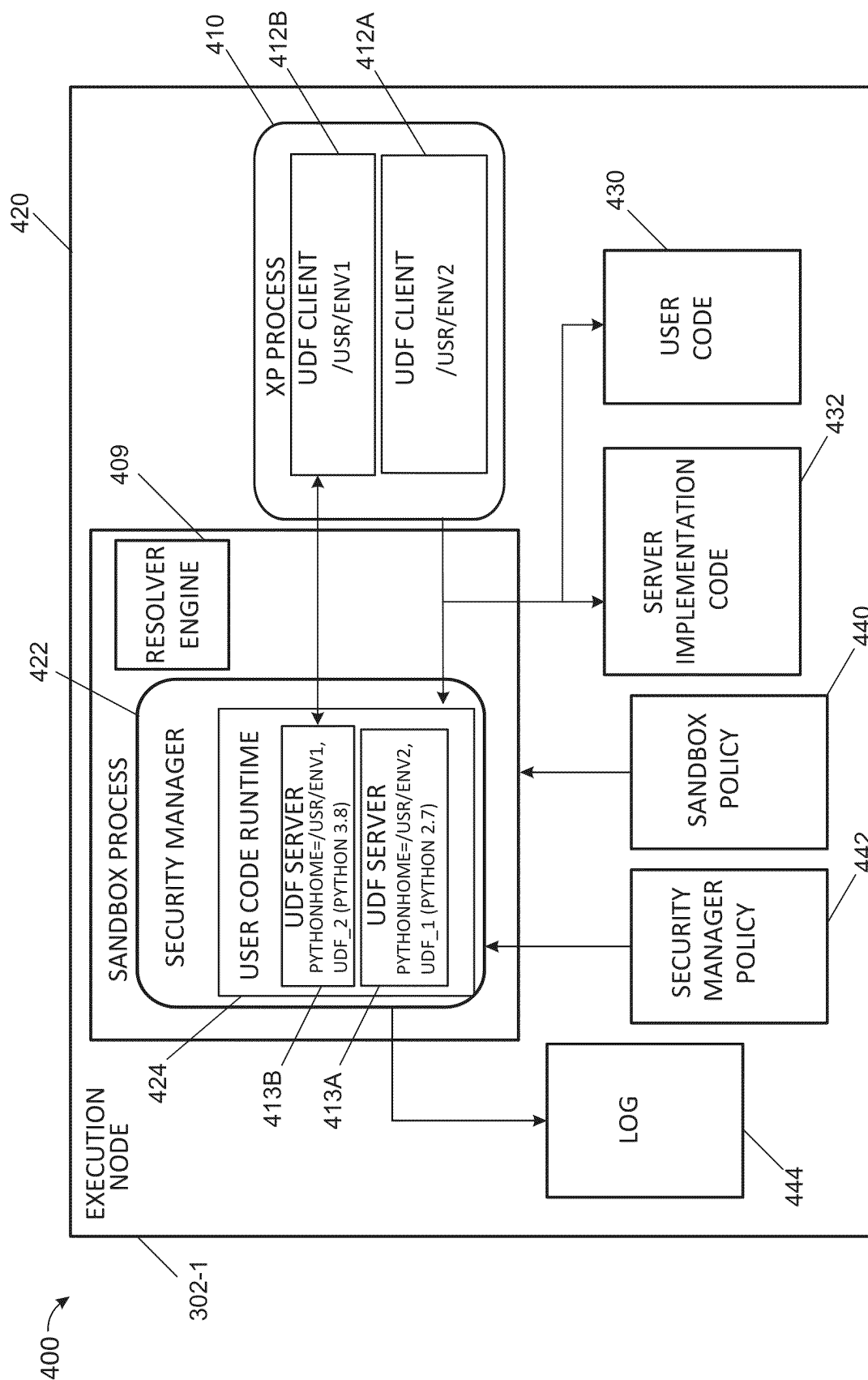
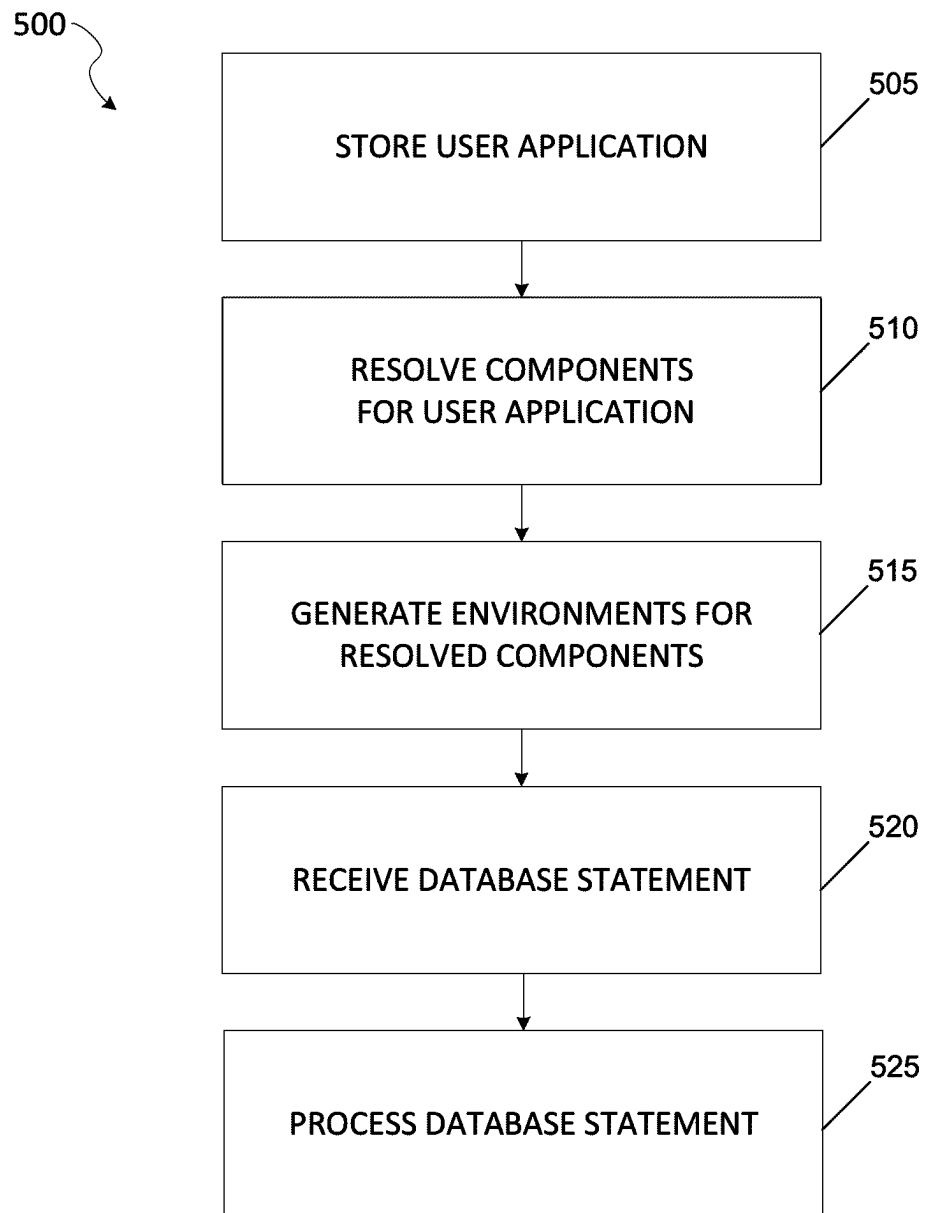
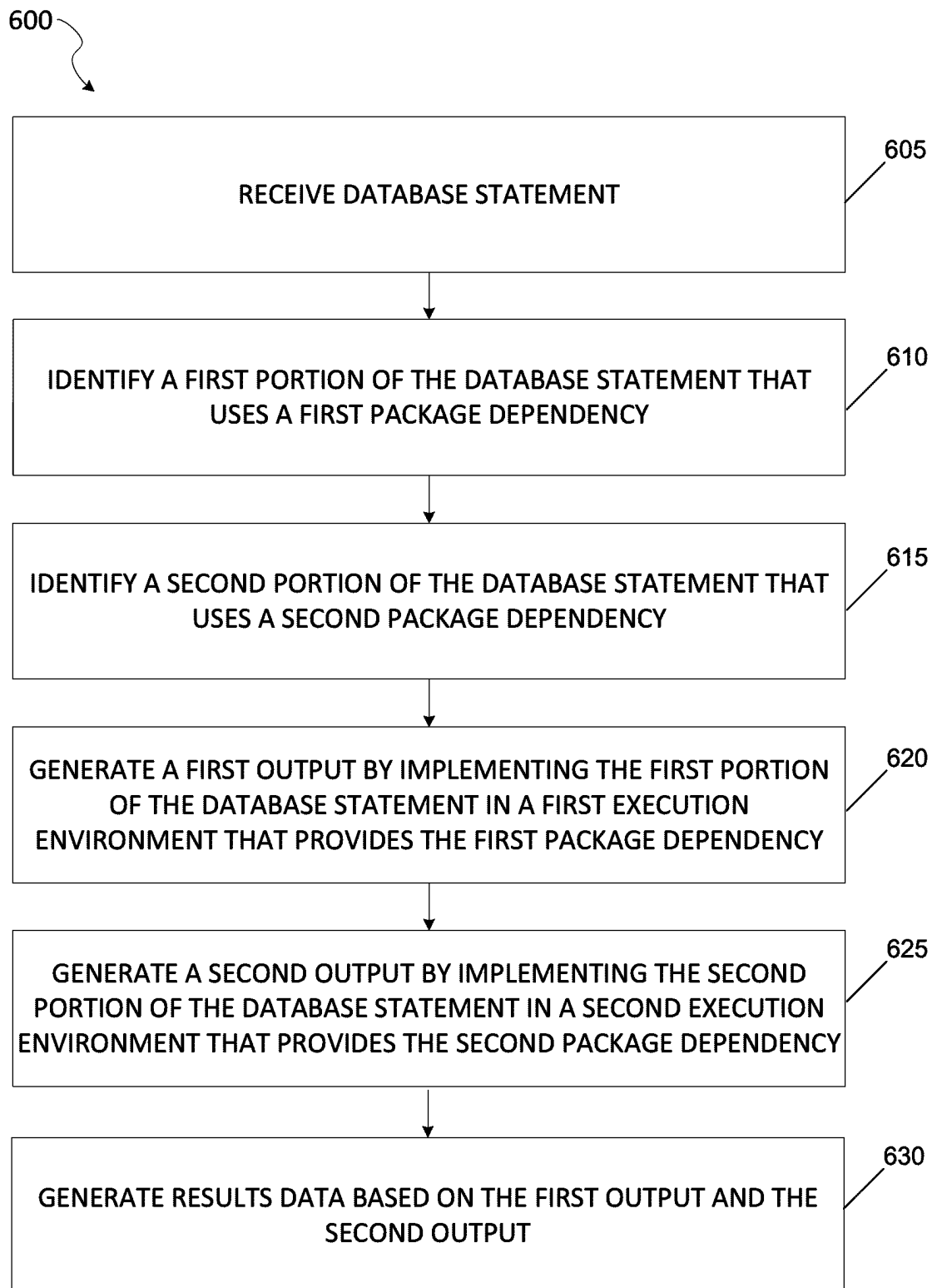


FIG. 4

*FIG. 5*

*FIG. 6*

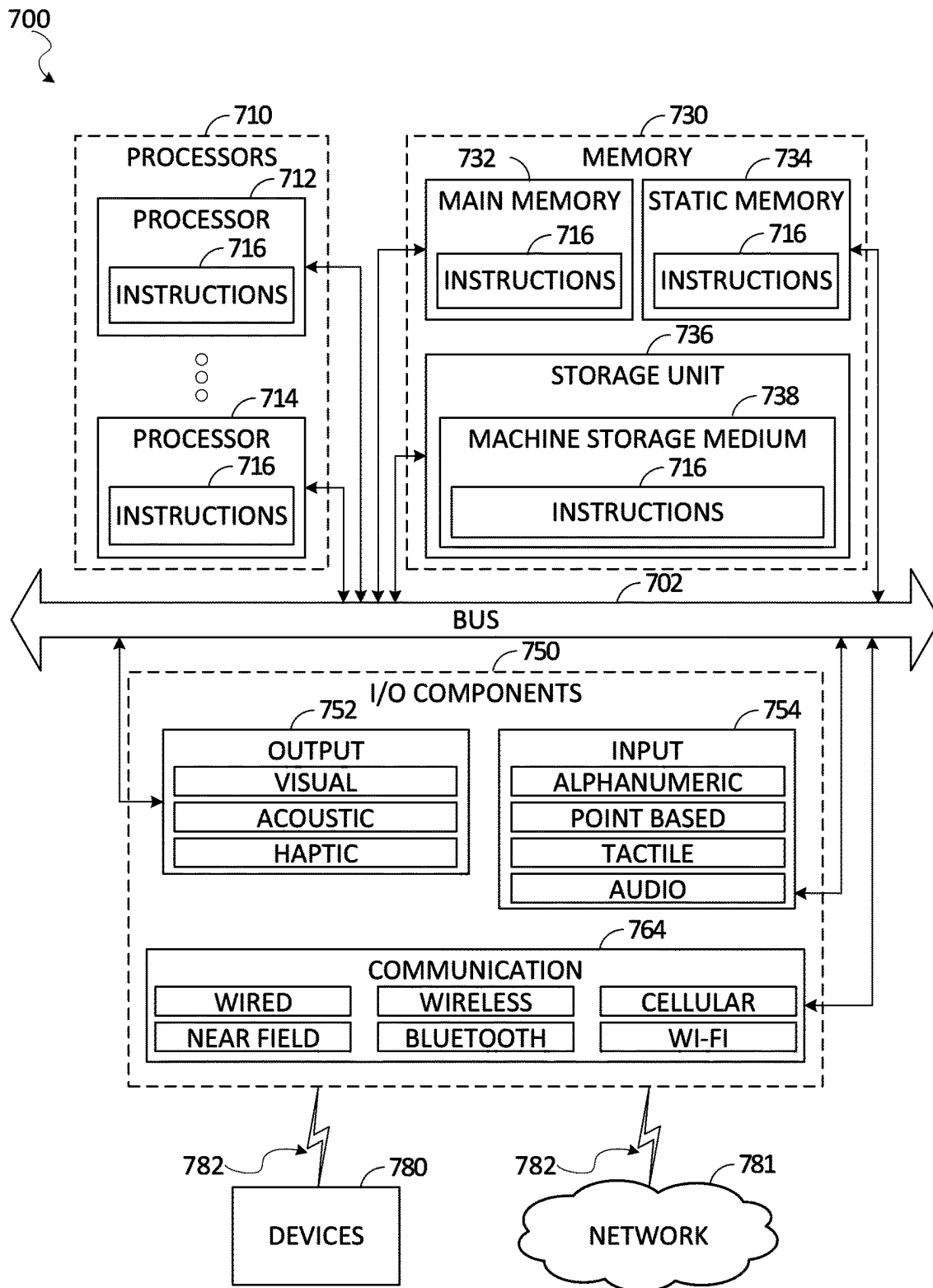


FIG. 7

1

MULTIPLE ENVIRONMENTS AND FUNCTIONS IN A DATABASE SYSTEM

CROSS REFERENCE TO RELATED APPLICATIONS

This application is a Continuation of U.S. patent application Ser. No. 18/051,483, filed on Oct. 31, 2022, which claims the benefit of priority of U.S. Provisional Application No. 63/268,583, filed on Feb. 25, 2022, the contents of which are incorporated herein by reference in their entireties.

TECHNICAL FIELD

Embodiments of the disclosure relate generally to a network-based database system and, more specifically, to implementing database applications in a distributed database system.

BACKGROUND

A user application can be provided for execution on a database system. Various components and libraries that are used by the application can be difficult to manage within the operational environment of the database system.

BRIEF DESCRIPTION OF THE DRAWINGS

The present disclosure will be understood more fully from the detailed description given below and from the accompanying drawings of various embodiments of the disclosure.

FIG. 1 illustrates an example computing environment that includes a network-based database system in communication with a cloud storage platform, in accordance with some embodiments of the present disclosure.

FIG. 2 is a block diagram illustrating components of a compute service manager, in accordance with some embodiments of the present disclosure.

FIG. 3 is a block diagram illustrating components of an execution platform, in accordance with some embodiments of the present disclosure.

FIG. 4 is a computing environment conceptually illustrating an example software architecture executing user-defined functions (UDFs) in multiple environments on a given execution node of the execution platform, in accordance with some embodiments of the present disclosure.

FIG. 5 shows a flow diagram of a method of implementing user-defined functions (UDFs) in multiple environments on a distributed database, in accordance with some embodiments of the present disclosure.

FIG. 6 is a flow diagram of a method for processing a database statement in multiple environments on a distributed database, in accordance with some embodiments of the present disclosure.

FIG. 7 illustrates a diagrammatic representation of a machine in the form of a computer system within which a set of instructions may be executed for causing the machine to perform any one or more of the methodologies discussed herein, in accordance with some embodiments of the present disclosure.

DETAILED DESCRIPTION

Reference will now be made in detail to specific example embodiments for carrying out the inventive subject matter. Examples of these specific embodiments are illustrated in

2

the accompanying drawings, and specific details are set forth in the following description in order to provide a thorough understanding of the subject matter. It will be understood that these examples are not intended to limit the scope of the claims to the illustrated embodiments. On the contrary, they are intended to cover such alternatives, modifications, and equivalents as may be included within the scope of the disclosure.

As discussed, it can be difficult to implement different programming language environments in a distributed database. As an example, Python is a popular language for data science and machine learning (ML). Python data science and ML applications can require different dependencies to function properly in a distributed database environment (e.g., virtual warehouses). One concern in implementing Python in a distributed database environment is dependency management. Dependencies include the software packages that are used by a given application (e.g., Python numpy) that must be installed in order for the application to work as intended and avoid runtime errors. One approach is to require end-users to upload and manage all the required packages, however this can be problematic because a given program language's versioning (e.g., Python versioning) can be unorganized and difficult to manage. Managing all the dependencies in this approach can result in negative development user experience (e.g., extreme frustration encountered by end-users when installed software packages have dependencies on specific versions of other software packages). For instance, the dependency issue arises when several packages have dependencies on the same shared packages or libraries, but they depend on different and incompatible versions of the shared packages. If the shared package or library can only be installed in a single version, the user may need to address the problem by obtaining newer or older versions of the dependent packages. This, in turn, may break other dependencies and push the problem to another set of packages. Furthermore, requiring users to install and manage hundreds of packages is insecure, cumbersome, and error prone. Another approach is to only integrate a small set of dependencies out of the box on the distributed database, however this sets a hard limit on application functionality, and users will not be able to tap into the full python ecosystem.

To address the foregoing issues, a database dependency resolver system is implemented to create one or more execution environments that are configured for different types of dependencies (e.g., programming language versions, CPU requirements). The database dependency resolver system comprises a resolver engine that resolves different dependencies of a given programming language application that is provided by the end-user for execution in the distributed database. In some example embodiments, the resolver engine generates a configuration file (e.g., yaml) that specifies which packages are to be installed for a given user application (e.g., Python application).

In some example embodiments, the database dependency resolver system comprises a repository of packages (e.g., full Python packages repository) and once the different packages for a given application are resolved, the database dependency resolver system retrieves them from the repository and configures one or more execution environments. The execution environments are configured for different versions of the programming language (e.g., Python version 2.7 in a first environment, Python version 3.8 in a second environment), in accordance with some example embodiments. A given user may have a UDF in their application that is written in Python 2.7 which cannot be upgraded to Python

3.8, but other UDFs are written in Python 3.8. In the improved approach that implements the database dependency resolver system, the user does not manage the different packages nor installs different versions. Instead, the user issues a database statement (e.g., query) that causes execution of the different environments. For example, a query such as “Select python_38_func(python_27_func())” in which the results from the Python 2.7 UDF are passed to the Python 3.8 UDF.

In some embodiments, the environments can be associated with repository versions that allow a user to recreate an environment in the same state as was originally created at a later time. For example, a user may specify creating a Python environment that supports numpy versions 1.2 or greater. Since an exact version is not specified, future environments created based on this specification could contain newer builds, which is not ideal if the user wants environments based on the same specification. To alleviate this issue, a repository version number is assigned to an environment when it is initially created to identify the state of the repository (e.g., included packaged) at the time it was created. A user may subsequently use the repository version number to recreate the same environment at a later time that includes the same set of packages.

Additionally, different environments can be created to manage different CPU architectures to enable changing the CPU architecture of the execution environment (e.g., where the Python code runs) without breaking the user code. For example, if the application is Python numpy, the system can initially provide an x86 build of numpy for execution on a x86 environment and then later switch to ARM build of numpy for execution in an ARM environment without breaking the user code. In this way, end-users need only import an application specifying packages in the user code and need not complete any installation or upgrading of the packages. Further, a given end-user can activate multiple environments for the different dependencies from a single database statement.

FIG. 1 illustrates an example computing environment 100 that includes a database system in the example form of a network-based database system 102, in accordance with some embodiments of the present disclosure. To avoid obscuring the inventive subject matter with unnecessary detail, various functional components that are not germane to conveying an understanding of the inventive subject matter have been omitted from FIG. 1. However, a skilled artisan will readily recognize that various additional functional components may be included as part of the computing environment 100 to facilitate additional functionality that is not specifically described herein. In other embodiments, the computing environment may comprise another type of network-based database system or a cloud data platform.

As shown, the computing environment 100 comprises the database system 102 in communication with a cloud storage platform 104 (e.g., AWS®, Microsoft Azure Blob Storage®, or Google Cloud Storage). The database system 102 is a network-based system used for reporting and analysis of integrated data from one or more disparate sources including one or more storage locations within the cloud storage platform 104. The cloud storage platform 104 comprises a plurality of computing machines and provides on-demand computer system resources such as data storage and computing power to the database system 102.

The database system 102 comprises a compute service manager 108, an execution platform 110, and one or more

metadata databases 112. The database system 102 hosts and provides data reporting and analysis services to multiple client accounts.

The compute service manager 108 coordinates and manages operations of the database system 102. The compute service manager 108 also performs query optimization and compilation as well as managing clusters of computing services that provide compute resources (also referred to as “virtual warehouses”). The compute service manager 108 can support any number of client accounts such as end users providing data storage and retrieval requests, system administrators managing the systems and methods described herein, and other components/devices that interact with compute service manager 108.

The compute service manager 108 is also in communication with a client device 114. The client device 114 corresponds to a user of one of the multiple client accounts supported by the database system 102. A user may utilize the client device 114 to submit data storage, retrieval, and analysis requests to the compute service manager 108.

The compute service manager 108 is also coupled to one or more metadata databases 112 that store metadata pertaining to various functions and aspects associated with the database system 102 and its users. For example, a metadata database 112 may include a summary of data stored in remote data storage systems as well as data available from a local cache. Additionally, a metadata database 112 may include information regarding how data is organized in remote data storage systems (e.g., the cloud storage platform 104) and the local caches. Information stored by a metadata database 112 allows systems and services to determine whether a piece of data needs to be accessed without loading or accessing the actual data from a storage device.

The compute service manager 108 is further coupled to the execution platform 110, which provides multiple computing resources that execute various data storage and data retrieval tasks. The execution platform 110 is coupled to cloud storage platform 104. The cloud storage platform 104 comprises multiple data storage devices 120-1 to 120-N. In some embodiments, the data storage devices 120-1 to 120-N are cloud-based storage devices located in one or more geographic locations. For example, the data storage devices 120-1 to 120-N may be part of a public cloud infrastructure or a private cloud infrastructure. The data storage devices 120-1 to 120-N may be hard disk drives (HDDs), solid state drives (SSDs), storage clusters, Amazon S3™ storage systems, or any other data storage technology. Additionally, the cloud storage platform 104 may include distributed file systems (such as Hadoop Distributed File Systems (HDFS)), object storage systems, and the like.

The execution platform 110 comprises a plurality of compute nodes. A set of processes on a compute node executes a query plan compiled by the compute service manager 108. The set of processes can include: a first process to execute the query plan; a second process to monitor and delete cache files using a least recently used (LRU) policy and implement an out of memory (OOM) error mitigation process; a third process that extracts health information from process logs and status to send back to the compute service manager 108; a fourth process to establish communication with the compute service manager 108 after a system boot; and a fifth process to handle all communication with a compute cluster for a given job provided by the compute service manager 108 and to communicate information back to the compute service manager 108 and other compute nodes of the execution platform 110.

In some embodiments, communication links between elements of the computing environment **100** are implemented via one or more data communication networks. These data communication networks may utilize any communication protocol and any type of communication medium. In some embodiments, the data communication networks are a combination of two or more data communication networks (or sub-Networks) coupled to one another. In alternate embodiments, these communication links are implemented using any type of communication medium and any communication protocol.

The compute service manager **108**, metadata database(s) **112**, execution platform **110**, and cloud storage platform **104** are shown in FIG. 1 as individual discrete components. However, each of the compute service manager **108**, metadata database(s) **112**, execution platform **110**, and cloud storage platform **104** may be implemented as a distributed system (e.g., distributed across multiple systems/platforms at multiple geographic locations). Additionally, each of the compute service manager **108**, metadata database(s) **112**, execution platform **110**, and cloud storage platform **104** can be scaled up or down (independently of one another) depending on changes to the requests received and the changing needs of the database system **102**. Thus, in the described embodiments, the database system **102** is dynamic and supports regular changes to meet the current data processing needs.

During typical operation, the database system **102** processes multiple jobs determined by the compute service manager **108**. These jobs are scheduled and managed by the compute service manager **108** to determine when and how to execute the job. For example, the compute service manager **108** may divide the job into multiple discrete tasks and may determine what data is needed to execute each of the multiple discrete tasks. The compute service manager **108** may assign each of the multiple discrete tasks to one or more nodes of the execution platform **110** to process the task. The compute service manager **108** may determine what data is needed to process a task and further determine which nodes within the execution platform **110** are best suited to process the task. Some nodes may have already cached the data needed to process the task and, therefore, be a good candidate for processing the task. Metadata stored in a metadata database **112** assists the compute service manager **108** in determining which nodes in the execution platform **110** have already cached at least a portion of the data needed to process the task. One or more nodes in the execution platform **110** process the task using data cached by the nodes and, if necessary, data retrieved from the cloud storage platform **104**. It is desirable to retrieve as much data as possible from caches within the execution platform **110** because the retrieval speed is typically much faster than retrieving data from the cloud storage platform **104**.

As shown in FIG. 1, the computing environment **100** separates the execution platform **110** from the cloud storage platform **104**. In this arrangement, the processing resources and cache resources in the execution platform **110** operate independently of the data storage devices **120-1** to **120-N** in the cloud storage platform **104**. Thus, the computing resources and cache resources are not restricted to specific data storage devices **120-1** to **120-N**. Instead, all computing resources and all cache resources may retrieve data from, and store data to, any of the data storage resources in the cloud storage platform **104**.

FIG. 2 is a block diagram illustrating components of the compute service manager **108**, in accordance with some embodiments of the present disclosure. As shown in FIG. 2,

the compute service manager **108** includes an access manager **202** and a credential management system **204** coupled to a data storage device **206** to access metadata, which is an example of the metadata database(s) **112**. Access manager **202** handles authentication and authorization tasks for the systems described herein. The credential management system **204** facilitates use of remote stored credentials to access external resources such as data resources in a remote storage device. As used herein, the remote storage devices may also be referred to as “persistent storage devices” or “shared storage devices.” For example, the credential management system **204** may create and maintain remote credential store definitions and credential objects (e.g., in the access metadata database **206**). A remote credential store definition identifies a remote credential store and includes access information to access security credentials from the remote credential store. A credential object identifies one or more security credentials using non-sensitive information (e.g., text strings) that are to be retrieved from a remote credential store for use in accessing an external resource. When a request invoking an external resource is received at run time, the credential management system **204** and access manager **202** use information stored in the access metadata database **206** (e.g., a credential object and a credential store definition) to retrieve security credentials used to access the external resource from a remote credential store.

A request processing service **208** manages received data storage requests and data retrieval requests (e.g., jobs to be performed on database data). For example, the request processing service **208** may determine the data to process a received query (e.g., a data storage request or data retrieval request). The data may be stored in a cache within the execution platform **110** or in a data storage device in cloud storage platform **104**.

A management console service **210** supports access to various systems and processes by administrators and other system managers. Additionally, the management console service **210** may receive a request to execute a job and monitor the workload on the system.

The compute service manager **108** also includes a job compiler **212**, a job optimizer **214**, and a job executor **216**. The job compiler **212** parses a job into multiple discrete tasks and generates the execution code for each of the multiple discrete tasks. The job optimizer **214** determines the best method to execute the multiple discrete tasks based on the data that needs to be processed. The job optimizer **214** also handles various data pruning operations and other data optimization techniques to improve the speed and efficiency of executing the job. The job executor **216** executes the execution code for jobs received from a queue or determined by the compute service manager **108**.

A job scheduler and coordinator **218** sends received jobs to the appropriate services or systems for compilation, optimization, and dispatch to the execution platform **110**. For example, jobs may be prioritized and then processed in that prioritized order. In an embodiment, the job scheduler and coordinator **218** determines a priority for internal jobs that are scheduled by the compute service manager **108** with other “outside” jobs such as user queries that may be scheduled by other systems in the database but may utilize the same processing resources in the execution platform **110**. In some embodiments, the job scheduler and coordinator **218** identifies or assigns particular nodes in the execution platform **110** to process particular tasks. A virtual warehouse manager **220** manages the operation of multiple virtual warehouses implemented in the execution platform **110**. For

example, the virtual warehouse manager **220** may generate query plans for executing received queries.

The database dependency resolver system **230** is configured to manage database dependencies and environments for a given programming language from a user application. Although in FIG. 2 the database dependency resolver system **230** is illustrated in the compute service manager **108**, in some example embodiments, instances of the database dependency resolver system **230** are implemented on execution nodes (XP nodes), e.g., for concurrent processing of portions of a queried dataset that is serialized and deserialized and processed using an external network service, as discussed in further detail below.

The database dependency resolver system **230** is implemented to create one or more execution environments that are configured for different types of dependencies (e.g., programming language versions, CPU requirements). The database dependency resolver system **230** comprises a resolver engine (not shown) that resolves different dependencies of a given programming language application that is provided by the end-user for execution in the distributed database. In some example embodiments, the resolver engine generates a configuration file (e.g., yaml) that specifies which packages are to be installed for a given user application (e.g., Python application). In some example embodiments, the database dependency resolver system **230** maintains a repository of packages (e.g., full Python packages repository) and once the different packages for a given application are resolved, the database dependency resolver system **230** retrieves them from the repository and configures one or more execution environments. The execution environments are configured for different versions of the programming language (e.g., Python version 2.7 in a first environment, Python version 3.8 in a second environment), in accordance with some example embodiments.

Additionally, different environments can be created to manage different CPU architectures to enable changing the CPU architecture of the execution environment (e.g., where the Python code runs) without breaking the user code. For example, if the application is Python numpy, the system can initially provide an x86 build of numpy for execution on a x86 environment and then later switch to an ARM build of numpy for execution in an ARM environment without breaking the user code. In this way, end-users need only import an application specifying packages in the user code, without having to perform any installation or upgrading of the packages. Further, a given end-user can activate multiple environments for the different dependencies from a single database statement. For example, a given user may have a UDF in their application that is written in Python 2.7 which cannot be upgraded to Python 3.8, but other UDFs are written in Python 3.8. Use of the database dependency resolver system **230** eliminates the burden to the user of managing the different packages and installing different versions. Instead, a user can simply issue a database statement (e.g., query) that causes execution of the different environments. For example, a user can define a query that includes a function using Python version 2.7 embedded within a function using Python version 3.8. To execute the function, the embedded function using Python version 2.7 is processed within the Python 2.7 UDF and the output is passed to the Python 3.8 UDF to complete processing of the function.

The functionality of the database dependency resolver system **230** is discussed in greater detail below in relation to FIGS. 4-6.

Additionally, the compute service manager **108** includes a configuration and metadata manager **222**, which manages the information related to the data stored in the remote data storage devices and in the local buffers (e.g., the buffers in execution platform **110**). The configuration and metadata manager **222** uses metadata to determine which data files need to be accessed to retrieve data for processing a particular task or job. A monitor and workload analyzer **224** oversees processes performed by the compute service manager **108** and manages the distribution of tasks (e.g., workload) across the virtual warehouses and execution nodes in the execution platform **110**. The monitor and workload analyzer **224** also redistributes tasks, as needed, based on changing workloads throughout the database system **102** and may further redistribute tasks based on a user (e.g., "external") query workload that may also be processed by the execution platform **110**. The configuration and metadata manager **222** and the monitor and workload analyzer **224** are coupled to a data storage device **226**. Data storage device **226** in FIG. 2 represents any data storage device within the database system **102**. For example, data storage device **226** may represent buffers in execution platform **110**, storage devices in cloud storage platform **104**, or any other storage device.

As described in embodiments herein, the compute service manager **108** validates all communication from an execution platform (e.g., the execution platform **110**) to validate that the content and context of that communication are consistent with the task(s) known to be assigned to the execution platform. For example, an instance of the execution platform executing a query A should not be allowed to request access to data-source D (e.g., data storage device **226**) that is not relevant to query A. Similarly, a given execution node (e.g., execution node **302-1**) may need to communicate with another execution node (e.g., execution node **302-2**), and should be disallowed from communicating with a third execution node (e.g., execution node **312-1**) and any such illicit communication can be recorded (e.g., in a log or other location). Also, the information stored on a given execution node is restricted to data relevant to the current query and any other data is unusable, rendered so by destruction or encryption where the key is unavailable.

FIG. 3 is a block diagram illustrating components of the execution platform **110**, in accordance with some embodiments of the present disclosure. As shown in FIG. 3, the execution platform **110** includes multiple virtual warehouses, including virtual warehouse **1**, virtual warehouse **2**, and virtual warehouse N. Each virtual warehouse includes multiple execution nodes that each include a data cache and a processor. The virtual warehouses can execute multiple tasks in parallel by using the multiple execution nodes. As discussed herein, the execution platform **110** can add new virtual warehouses and drop existing virtual warehouses in real-time based on the current processing needs of the systems and users. This flexibility allows the execution platform **110** to quickly deploy large amounts of computing resources when needed without being forced to continue paying for those computing resources when they are no longer needed. All virtual warehouses can access data from any data storage device (e.g., any storage device in cloud storage platform **104**).

Although each virtual warehouse shown in FIG. 3 includes three execution nodes, a particular virtual warehouse may include any number of execution nodes. Further, the number of execution nodes in a virtual warehouse is dynamic, such that new execution nodes are created when

additional demand is present, and existing execution nodes are deleted when they are no longer useful.

Each virtual warehouse is capable of accessing any of the data storage devices **120-1** to **120-N** shown in FIG. 1. Thus, the virtual warehouses are not necessarily assigned to a specific data storage device **120-1** to **120-N** and, instead, can access data from any of the data storage devices **120-1** to **120-N** within the cloud storage platform **104**. Similarly, each of the execution nodes shown in FIG. 3 can access data from any of the data storage devices **120-1** to **120-N**. In some embodiments, a particular virtual warehouse or a particular execution node may be temporarily assigned to a specific data storage device, but the virtual warehouse or execution node may later access data from any other data storage device.

In the example of FIG. 3, virtual warehouse **1** includes three execution nodes **302-1**, **302-2**, and **302-N**. Execution node **302-1** includes a cache **304-1** and a processor **306-1**. Execution node **302-2** includes a cache **304-2** and a processor **306-2**. Execution node **302-N** includes a cache **304-N** and a processor **306-N**. Each execution node **302-1**, **302-2**, and **302-N** is associated with processing one or more data storage and/or data retrieval tasks. For example, a virtual warehouse may handle data storage and data retrieval tasks associated with an internal service, such as a clustering service, a materialized view refresh service, a file compaction service, a storage procedure service, or a file upgrade service. In other implementations, a particular virtual warehouse may handle data storage and data retrieval tasks associated with a particular data storage system or a particular category of data.

Similar to virtual warehouse **1** discussed above, virtual warehouse **2** includes three execution nodes **312-1**, **312-2**, and **312-N**. Execution node **312-1** includes a cache **314-1** and a processor **316-1**. Execution node **312-2** includes a cache **314-2** and a processor **316-2**. Execution node **312-N** includes a cache **314-N** and a processor **316-N**. Additionally, virtual warehouse **3** includes three execution nodes **322-1**, **322-2**, and **322-N**. Execution node **322-1** includes a cache **324-1** and a processor **326-1**. Execution node **322-2** includes a cache **324-2** and a processor **326-2**. Execution node **322-N** includes a cache **324-N** and a processor **326-N**.

In some embodiments, the execution nodes shown in FIG. 3 are stateless with respect to the data being cached by the execution nodes. For example, these execution nodes do not store or otherwise maintain state information about the execution node, or the data being cached by a particular execution node. Thus, in the event of an execution node failure, the failed node can be transparently replaced by another node. Since there is no state information associated with the failed execution node, the new (replacement) execution node can easily replace the failed node without concern for recreating a particular state.

Although the execution nodes shown in FIG. 3 each includes one data cache and one processor, alternate embodiments may include execution nodes containing any number of processors and any number of caches. Additionally, the caches may vary in size among the different execution nodes. The caches shown in FIG. 3 store, in the local execution node, data that was retrieved from one or more data storage devices in cloud storage platform **104**. Thus, the caches reduce or eliminate the bottleneck problems occurring in platforms that consistently retrieve data from remote storage systems. Instead of repeatedly accessing data from the remote storage devices, the systems and methods described herein access data from the caches in the execution nodes, which is significantly faster and avoids the

bottleneck problem discussed above. In some embodiments, the caches are implemented using high-speed memory devices that provide fast access to the cached data. Each cache can store data from any of the storage devices in the cloud storage platform **104**.

Further, the cache resources and computing resources may vary between different execution nodes. For example, one execution node may contain significant computing resources and minimal cache resources, making the execution node useful for tasks that require fast scanning of large resources. Another execution node may contain significant cache resources and minimal computing resources, making this execution node useful for tasks that require caching of large amounts of data. Yet another execution node may contain cache resources providing faster input-output operations, useful for tasks that require fast scanning of large amounts of data. In some embodiments, the cache resources and computing resources associated with a particular execution node are determined when the execution node is created, based on the expected tasks to be performed by the execution node.

Additionally, the cache resources and computing resources associated with a particular execution node may change over time based on changing tasks performed by the execution node. For example, an execution node may be assigned more processing resources if the tasks performed by the execution node become more processor-intensive. Similarly, an execution node may be assigned more cache resources if the tasks performed by the execution node require a larger cache capacity.

Although virtual warehouses **1**, **2**, and **N** are associated with the same execution platform **110**, the virtual warehouses may be implemented using multiple computing systems at multiple geographic locations. For example, virtual warehouse **1** can be implemented by a computing system at a first geographic location, while virtual warehouses **2** and **N** are implemented by another computing system at a second geographic location. In some embodiments, these different computing systems are cloud-based computing systems maintained by one or more different entities.

Additionally, each virtual warehouse is shown in FIG. 3 as having multiple execution nodes. The multiple execution nodes associated with each virtual warehouse may be implemented using multiple computing systems at multiple geographic locations. For example, an instance of virtual warehouse **1** implements execution nodes **302-1** and **302-2** on one computing platform at a geographic location and implements execution node **302-N** at a different computing platform at another geographic location. Selecting particular computing systems to implement an execution node may depend on various factors, such as the level of resources needed for a particular execution node (e.g., processing resource requirements and cache requirements), the resources available at particular computing systems, communication capabilities of networks within a geographic location or between geographic locations, and which computing systems are already implementing other execution nodes in the virtual warehouse.

Execution platform **110** is also fault tolerant. For example, if one virtual warehouse fails, that virtual warehouse is quickly replaced with a different virtual warehouse at a different geographic location.

A particular execution platform **110** may include any number of virtual warehouses. Additionally, the number of virtual warehouses in a particular execution platform is dynamic, such that new virtual warehouses are created when additional processing and/or caching resources are needed.

11

Similarly, existing virtual warehouses may be deleted when the resources associated with the virtual warehouse are no longer useful.

In some embodiments, the virtual warehouses may operate on the same data in cloud storage platform **104**, but each virtual warehouse has its own execution nodes with independent processing and caching resources. This configuration allows requests on different virtual warehouses to be processed independently and with no interference between the requests. This independent processing, combined with the ability to dynamically add and remove virtual warehouses, supports the addition of new processing capacity for new users without impacting the performance observed by the existing users.

FIG. 4 is a computing environment **400** conceptually illustrating an example software architecture executing a user-defined function (UDF) by a process running on a given execution node of the execution platform **110**, in accordance with some embodiments of the present disclosure.

As illustrated, the execution node **302-1** from the execution platform **110** includes an execution node process **410**, which in an embodiment is running on the processor **306-1** and can also utilize memory from the cache **304-1** (or another memory device or storage). As mentioned herein, a “process” or “computing process” can refer to an instance of a computer program that is being executed by one or more threads by an execution node or execution platform.

As previously described, the compute service manager **108** processes communications from the execution platform **110** to validate that the content and context of the communications are consistent with the task(s) known to be assigned to the execution platform **110**. For example, the execution platform **110** executing a query A is not allowed to request access to a particular data source (e.g., data storage device **226** or any one of the storage devices in the cloud storage platform **104**) that is not relevant to query A. In an example, the execution node **302-1** may need to communicate with a second execution node (e.g., execution node **302-2**), but the security mechanisms described herein can disallow communication with a third execution node (e.g., execution node **312-1**). Moreover, any such illicit communication can be recorded (e.g., in a log **444** or other location). Further, the information stored on a given execution node is restricted to data relevant to the current query and any other data is unusable by decryption or encryption where the key is unavailable.

The resolver engine **409** (e.g., pip, python Anaconda with conda) analyzes a user’s application and pulls packages from a repository managed by the database dependency resolver system **230**. For example, the resolver engine **409** analyzes the user’s application to identify packages and any associated dependencies needed to properly execute the application and then accesses the identified packages from the repository.

The execution node process **410** executes a UDF client, such as UDF client **412A** or UDF client **412B**, as illustrated in the example of FIG. 4. In some example embodiments, one or more clients or servers can be generated by the database dependency resolver system **230** according to a given user’s application. For example, a given user may have a UDF in their application that is written in Python 2.7 which they cannot upgrade to Python 3.8, but the rest of their UDFs are written in Python 3.8. The functionality provided by the database dependency resolver system **230** eliminates the need for the user to manage the different packages and/or install different versions. Instead, the user can issue a database statement (e.g., query) that directly runs the dif-

12

ferent environments, such as a query: “Select python_38_func (python_27 func())”. In some example embodiments, a Python 2.7 execution environment is configured in UDF server **413A** and a Python 3.8 UDF execution environment is configured in UDF server **413B**, and when the query is received, the output from the Python 2.7 UDF is passed to the Python 3.8 UDF. The Python 3.8 UDF then uses the output from the Python 2.7 UDF to generate result data that is returned to the user.

User code **430** may be provided as a file that includes code for one or more UDFs. Server implementation code **432**, in an embodiment, is a programming language file (e.g., python file, JAR file) that initiates a server which is responsible for receiving requests from the execution node process **410**, assigning worker threads to execute user code, and returning the results, among other types of server tasks.

In an implementation, an operation from a UDF (e.g., python based UDF) can be performed by a user code runtime **424** executing within a sandbox process **420** (e.g., UDF server processes, such as UDF server **413A** or UDF server **413B**). In an embodiment, the user code runtime **424** is implemented as a virtual machine. Since the user code runtime **424** executes in a separate process relative to the execution node process **410**, the risk of manipulating the execution node process **410** is reduced. Results of performing the operation, among other types of information or messages, can be stored in a log **444** for review and retrieval.

In an embodiment, the log **444** can be stored locally in memory at the execution node **302-1**, or at a separate location such as the cloud storage platform **104**. Moreover, such results can be returned from the user code runtime **424** to the UDF client (e.g., UDF client **412A**, UDF client **412B**) utilizing a high-performance protocol (e.g., without serialization or deserialization of data, without memory copies; operates on record batches without having to access individual columns, records or cells; utilizes efficient remote procedure call techniques and network protocol(s) for data transfer) for data transfer (e.g., distributed datasets) that further provides authentication and encryption of the data transfer. In an embodiment, the UDF client (e.g., UDF client **412A**, UDF client **412B**) uses a data transport mechanism that supports a network transfer of columnar data between the user code runtime **424** (and vice-versa).

Security Manager **422**, in an example, can prevent completion of an operation from a given UDF by throwing an exception (e.g., if the operation is not permitted), or returns (e.g., doing nothing) if the operation is permitted. In an implementation, the Security Manager **422** is implemented as a JAVA security manager object that allows applications to implement a security policy such as a security manager policy **442**, and enables an application to determine, before performing a possibly unsafe or sensitive operation, what the operation is and whether it is being attempted in a security context that allows the operation to be performed. The security manager policy **442** can be implemented as a file with permissions that the user code runtime **424** is granted. The application (e.g., UDF executed by the user code runtime **424**) therefore can allow or disallow the operation based at least in part on the security policy.

Sandbox process **420**, in an embodiment, is a sub-process (or separate process) from the execution node process **410**. A sub-process, in an embodiment, refers to a child process of a given parent process (e.g., in this example, the execution node process **410**). The sandbox process **420**, in an example, is a program that reduces the risk of security breaches by restricting the running environment of untrusted applica-

13

tions using security mechanisms such as namespaces and secure computing modes (e.g., using a system call filter to an executing process and all its descendants, thus reducing the attack surface of the kernel of a given operating system). Moreover, in an example, the sandbox process **420** is a lightweight process in comparison to the execution node process **410** and is optimized (e.g., closely coupled to security mechanisms of a given operating system kernel) to process a database query in a secure manner within the sandbox environment.

In an embodiment, the sandbox process **420** can utilize a virtual network connection in order to communicate with other components within the subject system. A specific set of rules can be configured for the virtual network connection with respect to other components of the subject system. For example, such rules for the virtual network connection can be configured for a particular UDF to restrict the locations (e.g., particular sites on the Internet or components that the UDF can communicate) that are accessible by operations performed by the UDF. Thus, in this example, the UDF can be denied access to particular network locations or sites on the Internet.

The sandbox process **420** can be understood as providing a constrained computing environment for a process (or processes) within the sandbox, where these constrained processes can be controlled and restricted to limit access to certain computing resources.

Examples of security mechanisms can include the implementation of namespaces in which each respective group of processes executing within the sandbox environment has access to respective computing resources (e.g., process IDs, hostnames, user IDs, file names, names associated with network access, and inter-process communication) that are not accessible to another group of processes (which may have access to a different group of resources not accessible by the former group of processes), other container implementations, and the like. By having the sandbox process **420** execute as a sub-process to the execution node process **410**, in some embodiments, latency in processing a given database query can be substantially reduced (e.g., a reduction in latency by a factor of 10× in some instances) in comparison with other techniques that may utilize a virtual machine solution by itself.

As further illustrated, the sandbox process **420** can utilize a sandbox policy **440** to enforce a given security policy. The sandbox policy **440** can be a file with information related to a configuration of the sandbox process **420** and details regarding restrictions, if any, and permissions for accessing and utilizing system resources. Example restrictions can include restrictions to network access, or file system access (e.g., remapping file system to place files in different locations that may not be accessible, other files can be mounted in different locations, and the like). The sandbox process **420** restricts the memory and processor (e.g., CPU) usage of the user code runtime **424**, ensuring that other operations on the same execution node can execute without running out of resources.

As mentioned above, the sandbox process **420** is a sub-process (or separate process) from the execution node process **410**, which in practice means that the sandbox process **420** resides in a separate memory space than the execution node process **410**. In an occurrence of a security breach in connection with the sandbox process **420** (e.g., by errant or malicious code from a given UDF), if arbitrary memory is accessed by a malicious actor, the data or information stored by the execution node process is protected.

14

Although the above discussion of FIG. **4** describes components that are implemented using JAVA (e.g., an object-oriented programming language), it is appreciated that the other programming languages (e.g., interpreted programming languages) are supported by the computing environment **400**. In an embodiment, PYTHON is supported for implementing and executing UDFs in the computing environment **400**. In this example, the user code runtime **424** can be replaced with a PYTHON interpreter for executing operations from UDFs (e.g., written in PYTHON) within the sandbox process **420**.

FIG. **5** shows a flow diagram of a method **500** of implementing UDFs in multiple environments on a distributed database, in accordance with some embodiments of the present disclosure. The method **500** may be embodied in computer-readable instructions for execution by one or more hardware components (e.g., one or more processors) such that the operations of the method **500** may be performed by components of network-based database system **102**. Accordingly, the method **500** is described below, by way of example with reference thereto. However, it shall be appreciated that the method **500** may be deployed on various other hardware configurations and is not intended to be limited to deployment within the network-based database system **102**.

Depending on the embodiment, an operation of the method **500** may be repeated in different ways or involve intervening operations not shown. Though the operations of the method **500** may be depicted and described in a certain order, the order in which the operations are performed may vary among embodiments, including performing certain operations in parallel or performing sets of operations in separate processes.

At operation **505**, the database dependency resolver system **230** stores a user application. For example, the user application may be a python application uploaded by a database user.

At operation **510**, the resolver engine **409** determines which dependencies the user application uses to function. For example, the resolver engine **409** may analyze the user application to determine which packages, such as Python versions, libraries, are used by the user application. The resolver engine **409** may generate a configuration file that specifies the identified packages, which may then be used to access the packages. For example, the packages listed in the configuration file may be accessed from repository of packages maintained by the database dependency resolver system **230**.

At operation **515**, the database dependency resolver system **230** generates execution environments for the resolved components of the application (e.g., UDF client **412A**, UDF client **412B**, UDF server **413A**, UDF server **413B**). For example, the database dependency resolver system **230** may use the packages accessed from the repository to generate the execution environments.

In some embodiments, the generated environments can be associated with repository versions that allow a user to subsequently recreate the environment in the same state at a later time. For example, a user may specify creating a Python environment that supports numpy versions 1.2 or greater. Since an exact version is not specified, future environments created based on this specification could contain newer builds, which is not ideal if the user wants environments based on the same specification. To alleviate this issue, a repository version number is assigned to an environment when it is initially created to identify the state of the repository (e.g., included packaged) at the time it was created. A user may subsequently use the repository version

15

number to recreate the same environment at a later time that includes the same set of packages.

At operation **520**, the network-based database system **102** receives a database statement (e.g., query) from an end-user operation of the client device **114**. The database statement may include different portions that specify use of different sets of dependencies. For example, the database statement may include a portion identifying use of different programming languages and/or different versions of a programming language (e.g., “Select python_38_func (python_27_func())”, or “Select UDF_1, UDF_2” where UDF_1 is resolved as a Python version 2.7 UDF and UDF_2 is resolved as a Python version 3.8 UDF). Alternatively, the database statement may specify the repository version assigned to the execution environment to be used to process the database statement or a portion thereof.

At operation **525**, the database dependency resolver system **230** processes the received database statement to generate results data (e.g., using the execution environment and packages generated at operation **515**). For example, the database dependency resolver system **230** may generate outputs for the different portions of the database statement using the execution environment corresponding to the portion of the database statement. An example of processing a database statement in multiple environments is described in relation to FIG. 6.

FIG. 6 is a flow diagram of a method **600** for processing a database statement in multiple environments on a distributed database, according to some example embodiments. The method **600** may be embodied in computer-readable instructions for execution by one or more hardware components (e.g., one or more processors) such that the operations of the method **600** may be performed by components of network-based database system **102**. Accordingly, the method **600** is described below, by way of example with reference thereto. However, it shall be appreciated that the method **600** may be deployed on various other hardware configurations and is not intended to be limited to deployment within the network-based database system **102**.

Depending on the embodiment, an operation of the method **600** may be repeated in different ways or involve intervening operations not shown. Though the operations of the method **600** may be depicted and described in a certain order, the order in which the operations are performed may vary among embodiments, including performing certain operations in parallel or performing sets of operations in separate processes.

At operation **605**, the network-based database system **102** receives a database statement from an end-user operation of the client device **114**. The database statement may include different portions that specify use of different sets of dependencies. For example, the database statement may include portions identifying use of different programming languages, different versions of a programming language, and/or the repository version associated with the execution environment to be used to process the database statement or a portion thereof.

Accordingly, at operation **610**, the database dependency resolver system **230** identifies a first portion of the database statement that uses a first package dependency and at operation **615**, the database dependency resolver system **230** identifies a second portion of the database statement that uses a second package dependency. The different portions of the database statement may be identified based on the text of the database statement. For example, the database statement may utilize syntax defining the specified programming language and/or versions of a programming language that are to

16

be used to process the portion of the database statement. The database dependency resolver system **230** may therefore analyze the text of the database statement to identify the different portions that utilize different package dependencies.

At operation **620**, the database dependency resolver system **230** generates a first output by implementing the first portion of the database statement in a first execution environment that provides the first package dependency.

At operation **625**, the database dependency resolver system **230** generates a second output by implementing the second portion of the database statement in a second execution environment that provides the second package dependency.

At operation **630**, the database dependency resolver system **230** generates results data based on the first output and the second output. For example, the database dependency resolver system **230** may generate the results data by aggregating the first output and the second output. As another example, the database dependency resolver system **230** may generate the results data by using one of the outputs at input when generating a subsequent output. For example, a user can define a query that includes a function using Python version 2.7 embedded within another function using Python version 3.8. To execute the query, the embedded function using Python version 2.7 is processed within the Python 2.7 UDF and the output is passed to the Python 3.8 UDF to complete processing of the query.

Described implementations of the subject matter can include one or more features, alone or in combination as illustrated below by way of example.

Example 1 is a method comprising: storing a user application on a distributed database system; identifying a plurality of package dependencies of the user application, the plurality of package dependencies comprising a first package dependency and a second package dependency; generating, by at least one hardware processor, a first execution environment that comprises the first package dependency; generating, by the at least one hardware processor, a second execution environment that comprises the second package dependency; receiving a database statement that includes a first portion using the first package dependency and a second portion using the second package dependency; and based on receiving the database statement, generating results data using the first execution environment and the second execution environment.

In Example 2, the subject matter of Example 1 includes, wherein generating the results data using the first execution environment and the second execution environment comprises: generating a first output by implementing the first portion of the database statement in the first execution environment; and generating a second output by implementing the second portion of the database statement in the second execution environment.

In Example 3, the subject matter of any of Examples 1-2 includes, wherein generating the results data using the first execution environment and the second execution environment further comprises: generating the results data based on the first output and the second output.

In Example 4, the subject matter of any of Examples 1-3 includes, wherein the second output is generated based on the first output.

In Example 5, the subject matter of any of Examples 1-4 includes, wherein identifying the plurality of package dependencies of the user application comprises: generating a configuration file that specifies the plurality of package dependencies of the user application.

17

In Example 6, the subject matter of any of Examples 1-5 includes, further comprising: accessing the plurality of package dependencies from a repository of packages based on the configuration file.

In Example 7, the subject matter of any of Examples 1-6 includes, wherein the first execution environment is configured for a first version of a programming language and the second execution environment is configured for a second version of the programming language.

In Example 8, the subject matter of any of Examples 1-7 includes, wherein the programming language is Python.

In Example 9, the subject matter of any of Examples 1-8 includes, wherein the first execution environment is configured for a first programming language and the second execution environment is configured for a second programming language.

In Example 10, the subject matter of any of Examples 1-9 includes, wherein the first execution environment is configured for a first computing architecture and the second execution environment is configured for a second computing architecture.

Example 11 is a system comprising: one or more computer processors; and one or more computer-readable mediums storing instructions that, when executed by the one or more computer processors, cause the system to perform operations comprising: storing a user application on a distributed database system; identifying a plurality of package dependencies of the user application, the plurality of package dependencies comprising a first package dependency and a second package dependency; generating a first execution environment that comprises the first package dependency; generating a second execution environment that comprises the second package dependency; receiving a database statement that includes a first portion using the first package dependency and a second portion using the second package dependency; and based on receiving the database statement, generating results data using the first execution environment and the second execution environment.

In Example 12, the subject matter of Example 11 includes, wherein generating the results data using the first execution environment and the second execution environment comprises: generating a first output by implementing the first portion of the database statement in the first execution environment; and generating a second output by implementing the second portion of the database statement in the second execution environment.

In Example 13, the subject matter of any of Examples 11-12 includes, wherein generating the results data using the first execution environment and the second execution environment further comprises: generating the results data based on the first output and the second output.

In Example 14, the subject matter of any of Examples 11-13 includes, wherein the second output is generated based on the first output.

In Example 15, the subject matter of any of Examples 11-14 includes, wherein identifying the plurality of package dependencies of the user application comprises: generating a configuration file that specifies the plurality of package dependencies of the user application.

In Example 16, the subject matter of any of Examples 11-15 includes, the operations further comprising: accessing the plurality of package dependencies from a repository of packages based on the configuration file.

In Example 17, the subject matter of any of Examples 11-16 includes, wherein the first execution environment is configured for a first version of a programming language and

18

the second execution environment is configured for a second version of the programming language.

In Example 18, the subject matter of any of Examples 11-17 includes, wherein the programming language is Python.

In Example 19, the subject matter of any of Examples 11-18 includes, wherein the first execution environment is configured for a first programming language and the second execution environment is configured for a second programming language.

In Example 20, the subject matter of any of Examples 11-19 includes, wherein the first execution environment is configured for a first computing architecture and the second execution environment is configured for a second computing architecture.

Example 21 is a computer-storage medium storing instructions that, when executed by one or more computer processors of one or more computing devices, cause the one or more computing devices to perform operations comprising: storing a user application on a distributed database system; identifying a plurality of package dependencies of the user application, the plurality of package dependencies comprising a first package dependency and a second package dependency; generating a first execution environment that comprises the first package dependency; generating a second execution environment that comprises the second package dependency; receiving a database statement that includes a first portion using the first package dependency and a second portion using the second package dependency; and based on receiving the database statement, generating results data using the first execution environment and the second execution environment.

In Example 22, the subject matter of Examples 21 includes, wherein generating the results data using the first execution environment and the second execution environment comprises: generating a first output by implementing the first portion of the database statement in the first execution environment; and generating a second output by implementing the second portion of the database statement in the second execution environment.

In Example 23, the subject matter of any of Examples 21-22 includes, wherein generating the results data using the first execution environment and the second execution environment further comprises: generating the results data based on the first output and the second output.

In Example 24, the subject matter of any of Examples 21-23 includes, wherein the second output is generated based on the first output.

In Example 25, the subject matter of any of Examples 21-24 includes, wherein identifying the plurality of package dependencies of the user application comprises: generating a configuration file that specifies the plurality of package dependencies of the user application.

In Example 26, the subject matter of any of Examples 21-25 includes, the operations further comprising: accessing the plurality of package dependencies from a repository of packages based on the configuration file.

In Example 27, the subject matter of any of Examples 21-26 includes, wherein the first execution environment is configured for a first version of a programming language and the second execution environment is configured for a second version of the programming language.

In Example 28, the subject matter of any of Examples 21-27 includes, wherein the programming language is Python.

In Example 29, the subject matter of any of Examples 21-28 includes, wherein the first execution environment is

configured for a first programming language and the second execution environment is configured for a second programming language.

In Example 30, the subject matter of any of Examples 21-29 includes, wherein the first execution environment is configured for a first computing architecture and the second execution environment is configured for a second computing architecture.

FIG. 7 illustrates a diagrammatic representation of a machine 700 in the form of a computer system within which a set of instructions may be executed for causing the machine 700 to perform any one or more of the methodologies discussed herein, according to an example embodiment. Specifically, FIG. 7 shows a diagrammatic representation of the machine 700 in the example form of a computer system, within which instructions 716 (e.g., software, a program, an application, an applet, an app, or other executable code) for causing the machine 700 to perform any one or more of the methodologies discussed herein may be executed. For example, the instructions 716 may cause the machine 700 to execute any one or more operations of the methods 500 and 600. As another example, the instructions 716 may cause the machine 700 to implement one or more portions of the methods of FIG. 5. In this way, the instructions 716 transform a general, non-programmed machine into a particular machine 700 (e.g., the compute service manager 108 or a node in the execution platform 110) that is specially configured to carry out any one of the described and illustrated functions in the manner described herein.

In alternative embodiments, the machine 700 operates as a standalone device or may be coupled (e.g., networked) to other machines. In a networked deployment, the machine 700 may operate in the capacity of a server machine or a client machine in a server-client network environment, or as a peer machine in a peer-to-peer (or distributed) network environment. The machine 700 may comprise, but not be limited to, a server computer, a client computer, a personal computer (PC), a tablet computer, a laptop computer, a netbook, a smart phone, a mobile device, a network router, a network switch, a network bridge, or any machine capable of executing the instructions 716, sequentially or otherwise, that specify actions to be taken by the machine 700. Further, while only a single one of the machines 700 is illustrated, the term “machine” shall also be taken to include a collection of machines 700 that individually or jointly execute the instructions 716 to perform any one or more of the methodologies discussed herein.

The machine 700 includes multi-core processors 710, memory 730, and input/output (I/O) components 750 configured to communicate with each other such as via a bus 702. In an example embodiment, the multi-core processors 710 (e.g., a central processing unit (CPU), a reduced instruction set computing (RISC) processor, a complex instruction set computing (CISC) processor, a graphics processing unit (GPU), a digital signal processor (DSP), an application-specific integrated circuit (ASIC), a radio-frequency integrated circuit (RFIC), another processor, or any suitable combination thereof) may include, for example, a processor 712 and a processor 714 that may execute the instructions 716. The term “processor” is intended to include multi-core processors 710 that may comprise two or more independent processors (sometimes referred to as “cores”) that may execute instructions 716 contemporaneously. Although FIG. 7 shows multi-core processors 710, the machine 700 may include a single processor with a single core, a single processor with multiple cores (e.g., a multi-core processor),

multiple processors with a single core, multiple processors with multiple cores, or any combination thereof.

The memory 730 may include a main memory 732, a static memory 734, and a storage unit 736, all accessible to the multi-core processors 710 such as via the bus 702. The main memory 732, the static memory 734, and the storage unit 736 store the instructions 716 embodying any one or more of the methodologies or functions described herein. The instructions 716 may also reside, completely or partially, within the main memory 732, within the static memory 734, within machine storage medium 738 of the storage unit 736, within at least one of the multi-core processors 710 (e.g., within the processor's cache memory), or any suitable combination thereof, during execution thereof by the machine 700.

The I/O components 750 include components to receive input, provide output, produce output, transmit information, exchange information, capture measurements, and so on. The specific I/O components 750 that are included in a particular machine 700 will depend on the type of machine. For example, portable machines such as mobile phones will likely include a touch input device or other such input mechanisms, while a headless server machine will likely not include such a touch input device. It will be appreciated that the I/O components 750 may include many other components that are not shown in FIG. 7. The I/O components 750 are grouped according to functionality merely for simplifying the following discussion and the grouping is in no way limiting. In various example embodiments, the I/O components 750 may include output components 752 and input components 754. The output components 752 may include visual components (e.g., a display such as a plasma display panel (PDP), a light emitting diode (LED) display, a liquid crystal display (LCD), a projector, or a cathode ray tube (CRT)), acoustic components (e.g., speakers), other signal generators, and so forth. The input components 754 may include alphanumeric input components (e.g., a keyboard, a touch screen configured to receive alphanumeric input, a photo-optical keyboard, or other alphanumeric input components), point-based input components (e.g., a mouse, a touchpad, a trackball, a joystick, a motion sensor, or another pointing instrument), tactile input components (e.g., a physical button, a touch screen that provides location and/or force of touches or touch gestures, or other tactile input components), audio input components (e.g., a microphone), and the like.

Communication may be implemented using a wide variety of technologies. The I/O components 750 may include communication components 764 operable to couple the machine 700 to a network 781 or devices 780 via a couplings 782, respectively. For example, the communication components 764 may include a network interface component or another suitable device to interface with the network 781. In further examples, the communication components 764 may include wired communication components, wireless communication components, cellular communication components, and other communication components to provide communication via other modalities. The devices 780 may be another machine or any of a wide variety of peripheral devices (e.g., a peripheral device coupled via a universal serial bus (USB)). For example, as noted above, the machine 700 may correspond to any one of the compute service manager 108 or the execution platform 110, and the devices 780 may include the client device 114 or any other computing device described herein as being in communication with the database system 102 or the cloud storage platform 104.

The various memories (e.g., **730**, **732**, **734**, and/or memory of the processor(s) **710** and/or the storage unit **736**) may store one or more sets of instructions **716** and data structures (e.g., software) embodying or utilized by any one or more of the methodologies or functions described herein. These instructions **716**, when executed by the processor(s) **710**, cause various operations to implement the disclosed embodiments.

As used herein, the terms “machine-storage medium,” “device-storage medium,” and “computer-storage medium” mean the same thing and may be used interchangeably in this disclosure. The terms refer to a single or multiple storage devices and/or media (e.g., a centralized or distributed database, and/or associated caches and servers) that store executable instructions and/or data. The terms shall accordingly be taken to include, but not be limited to, solid-state memories, and optical and magnetic media, including memory internal or external to processors. Specific examples of machine-storage media, computer-storage media, and/or device-storage media include non-volatile memory, including by way of example semiconductor memory devices, e.g., erasable programmable read-only memory (EPROM), electrically erasable programmable read-only memory (EEPROM), field-programmable gate arrays (FPGAs), and flash memory devices; magnetic disks such as internal hard disks and removable disks; magneto-optical disks; and CD-ROM and DVD-ROM disks. The terms “machine-storage media,” “computer-storage media,” and “device-storage media” specifically exclude carrier waves, modulated data signals, and other such media, at least some of which are covered under the term “signal medium” discussed below.

In various example embodiments, one or more portions of the network **781** may be an ad hoc network, an intranet, an extranet, a virtual private network (VPN), a local-area network (LAN), a wireless LAN (WLAN), a wide-area network (WAN), a wireless WAN (WWAN), a metropolitan-area network (MAN), the Internet, a portion of the Internet, a portion of the public switched telephone network (PSTN), a plain old telephone service (POTS) network, a cellular telephone network, a wireless network, a Wi-Fi® network, another type of network, or a combination of two or more such networks. For example, the network **781** or a portion of the network **781** may include a wireless or cellular network, and the coupling **782** may be a Code Division Multiple Access (CDMA) connection, a Global System for Mobile communications (GSM) connection, or another type of cellular or wireless coupling. In this example, the coupling **782** may implement any of a variety of types of data transfer technology, such as Single Carrier Radio Transmission Technology (1×RTT), Evolution-Data Optimized (EVDO) technology, General Packet Radio Service (GPRS) technology, Enhanced Data rates for GSM Evolution (EDGE) technology, third Generation Partnership Project (3GPP) including 3G, fourth generation wireless (4G) networks, Universal Mobile Telecommunications System (UMTS), High-Speed Packet Access (HSPA), Worldwide Interoperability for Microwave Access (WiMAX), Long Term Evolution (LTE) standard, others defined by various standard-setting organizations, other long-range protocols, or other data transfer technology.

The instructions **716** may be transmitted or received over the network **781** using a transmission medium via a network interface device (e.g., a network interface component included in the communication components **764**) and utilizing any one of a number of well-known transfer protocols (e.g., hypertext transfer protocol (HTTP)). Similarly, the

instructions **716** may be transmitted or received using a transmission medium via the coupling **782** (e.g., a peer-to-peer coupling) to the devices **780**. The terms “transmission medium” and “signal medium” mean the same thing and may be used interchangeably in this disclosure. The terms “transmission medium” and “signal medium” shall be taken to include any intangible medium that is capable of storing, encoding, or carrying the instructions **716** for execution by the machine **700**, and include digital or analog communications signals or other intangible media to facilitate communication of such software. Hence, the terms “transmission medium” and “signal medium” shall be taken to include any form of modulated data signal, carrier wave, and so forth. The term “modulated data signal” means a signal that has one or more of its characteristics set or changed in such a manner as to encode information in the signal.

The terms “machine-readable medium,” “computer-readable medium,” and “device-readable medium” mean the same thing and may be used interchangeably in this disclosure. The terms are defined to include both machine-storage media and transmission media. Thus, the terms include both storage devices/media and carrier waves/modulated data signals.

The various operations of example methods described herein may be performed, at least partially, by one or more processors that are temporarily configured (e.g., by software) or permanently configured to perform the relevant operations. Similarly, the methods described herein may be at least partially processor-implemented. For example, at least some of the operations of the methods **500** and **600** may be performed by one or more processors. The performance of certain of the operations may be distributed among the one or more processors, not only residing within a single machine, but also deployed across a number of machines. In some example embodiments, the processor or processors may be located in a single location (e.g., within a home environment, an office environment, or a server farm), while in other embodiments the processors may be distributed across a number of locations. In view of the disclosure above, various examples are set forth below. It should be noted that one or more features of an example, taken in isolation or combination, should be considered within the disclosure of this application.

Although the embodiments of the present disclosure have been described with reference to specific example embodiments, it will be evident that various modifications and changes may be made to these embodiments without departing from the broader scope of the inventive subject matter. Accordingly, the specification and drawings are to be regarded in an illustrative rather than a restrictive sense. The accompanying drawings that form a part hereof show, by way of illustration, and not of limitation, specific embodiments in which the subject matter may be practiced. The embodiments illustrated are described in sufficient detail to enable those skilled in the art to practice the teachings disclosed herein. Other embodiments may be used and derived therefrom, such that structural and logical substitutions and changes may be made without departing from the scope of this disclosure. This Detailed Description, therefore, is not to be taken in a limiting sense, and the scope of various embodiments is defined only by the appended claims, along with the full range of equivalents to which such claims are entitled.

Such embodiments of the inventive subject matter may be referred to herein, individually and/or collectively, by the term “invention” merely for convenience and without intending to voluntarily limit the scope of this application to

23

any single invention or inventive concept if more than one is in fact disclosed. Thus, although specific embodiments have been illustrated and described herein, it should be appreciated that any arrangement calculated to achieve the same purpose may be substituted for the specific embodiments shown. This disclosure is intended to cover any and all adaptations or variations of various embodiments. Combinations of the above embodiments, and other embodiments not specifically described herein, will be apparent, to those of skill in the art, upon reviewing the above description.

In this document, the terms “a” or “an” are used, as is common in patent documents, to include one or more than one, independent of any other instances or usages of “at least one” or “one or more.” In this document, the term “or” is used to refer to a nonexclusive or, such that “A or B” includes “A but not B,” “B but not A,” and “A and B,” unless otherwise indicated. In the appended claims, the terms “including” and “in which” are used as the plain-English equivalents of the respective terms “comprising” and “wherein.” Also, in the following claims, the terms “including” and “comprising” are open-ended; that is, a system, device, article, or process that includes elements in addition to those listed after such a term in a claim is still deemed to fall within the scope of that claim.

What is claimed is:

1. A method comprising:

storing a user application on a database system;

identifying, by at least one hardware processor, a plurality of package dependencies of the user application from a repository of packages, the plurality of package dependencies comprising a first package dependency and a second package dependency;

receiving, from a user of the database system, a database statement comprises a first portion using the first package dependency and a second portion using the second package dependency;

analyzing the database statement to identify the first portion of the database statement based on a syntax defining a programming language, and a first version of the programming language and to identify the second portion of the database statement based on the syntax defining the programming language and a second version of the programming language;

after the analyzing of the database statement, retrieving the first package dependency and the second package dependency from the repository of the packages;

after retrieving the first package dependency, generating, on the database system, a first execution environment that comprises the first package dependency and that executes user-defined functions based on the first version of the programming language;

after retrieving the second package dependency, generating, on the database system, a second execution environment that comprises the second package dependency and that executes user-defined functions based on the second version of the programming language; and

generating results data based on the database statement by the first execution environment processing the first portion using the first package dependency and the first version of the programming language, and by the second execution environment processing the second portion using the second package dependency and the second version of the programming language.

24

2. The method of claim 1,

wherein the first execution environment and the second execution environment are generated on separate nodes of the database system.

3. The method of claim 1, further comprising:

assigning a first repository version number to the first execution environment;

assigning a second repository version number to the second execution environment; and

enabling recreation of the first execution environment based on the first repository version number and the second execution environment based on the second repository version number at a later time.

4. The method of claim 1, wherein the first package dependency and the second package dependency are associated with different computing architectures, the first execution environment configured to support a first computing architecture and the second execution environment configured to support a second computing architecture.

5. The method of claim 1, further comprising:

managing, via the database system, the repository of the packages, wherein the managing further comprises:

logging the generating of the first execution environment;

logging the generating of the second execution environment;

retrieving the first package dependency and the second package dependency from the repository of the packages;

logging the retrieving of the first package dependency; and

logging the retrieving of the second package dependency.

6. The method of claim 1, wherein generating the results data further comprises:

aggregating a first output from the first execution environment; and

aggregating a second output from the second execution environment.

7. The method of claim 6, wherein generating the results data further comprises:

passing the first output from the first execution environment as input to the second execution environment.

8. The method of claim 6, further comprising:

generating the first output based on execution of a user-defined function in the first execution environment; and

generating the second output based on the execution of the user-defined function in the second execution environment.

9. A system comprising:

one or more computer processors; and

one or more computer-readable mediums storing instructions that, when executed by the one or more computer processors, cause the system to perform operations comprising:

storing a user application on a database system;

identifying, by at least one hardware processor, a plurality of package dependencies of the user application from a repository of packages, the plurality of package dependencies comprising a first package dependency and a second package dependency;

receiving, from a user of the database system, a database statement comprises a first portion using the first package dependency and a second portion using the second package dependency;

analyzing the database statement to identify the first portion of the database statement based on a syntax defining a programming language, and a first version of the programming language and to identify the

25

second portion of the database statement based on the syntax defining the programming language and a second version of the programming language;
 after the analyzing of the database statement, retrieving the first package dependency and the second package dependency from the repository of the packages;
 after retrieving the first package dependency, generating, on the database system, a first execution environment that comprises the first package dependency and that executes user-defined functions based on the first version of the programming language;
 after retrieving the second package dependency, generating, on the database system, a second execution environment that comprises the second package dependency and that executes user-defined functions based on the second version of the programming language; and
 generating results data based on the database statement by the first execution environment processing the first portion using the first package dependency and the first version of the programming language, and by the second execution environment processing the second portion using the second package dependency and the second version of the programming language.

10. The system of claim 9, wherein the first execution environment and the second execution environment are generated on separate nodes of the database system.

11. The system of claim 9, the operations further comprising:
 assigning a first repository version number to the first execution environment;
 assigning a second repository version number to the second execution environment; and
 enabling recreation of the first execution environment based on the first repository version number and the second execution environment based on the second repository version number at a later time.

12. The system of claim 9, wherein the first package dependency and the second package dependency are associated with different computing architectures, the first execution environment configured to support a first computing architecture and the second execution environment configured to support a second computing architecture.

13. The system of claim 9, the operations further comprising:
 managing, via the database system, the repository of the packages, wherein the managing further comprises:
 logging the generating of the first execution environment;
 logging the generating of the second execution environment;
 retrieving the first package dependency and the second package dependency from the repository of the packages;
 logging the retrieving of the first package dependency; and
 logging the retrieving of the second package dependency.

14. The system of claim 9, wherein generating the results data further comprises:
 aggregating a first output from the first execution environment; and
 aggregating a second output from the second execution environment.

15. The system of claim 14, wherein generating the results data further comprises:

26

passing the first output from the first execution environment as input to the second execution environment.

16. The system of claim 14, the operations further comprising:
 generating the first output based on execution of a user-defined function in the first execution environment; and
 generating the second output based on the execution of the user-defined function in the second execution environment.

17. A machine-storage medium embodying instructions that, when executed by a machine, cause the machine to perform operations comprising:
 storing a user application on a database system;
 identifying, by at least one hardware processor, a plurality of package dependencies of the user application from a repository of packages, the plurality of package dependencies comprising a first package dependency and a second package dependency;
 receiving, from a user of the database system, a database statement comprising a first portion using the first package dependency and a second portion using the second package dependency;
 analyzing the database statement to identify the first portion of the database statement based on a syntax defining a programming language, and a first version of the programming language and to identify the second portion of the database statement based on the syntax defining the programming language and a second version of the programming language;
 after the analyzing of the database statement, retrieving the first package dependency and the second package dependency from the repository of the packages;
 after retrieving the first package dependency, generating, on the database system, a first execution environment that comprises the first package dependency and that executes user-defined functions based on the first version of the programming language;
 after retrieving the second package dependency, generating, on the database system, a second execution environment that comprises the second package dependency and that executes user-defined functions based on the second version of the programming language; and
 generating results data based on the database statement by the first execution environment processing the first portion using the first package dependency and the first version of the programming language, and by the second execution environment processing the second portion using the second package dependency and the second version of the programming language.

18. The machine-storage medium of claim 17, wherein the first execution environment and the second execution environment are generated on separate nodes of the database system.

19. The machine-storage medium of claim 17, the operations further comprising:
 assigning a first repository version number to the first execution environment;
 assigning a second repository version number to the second execution environment; and
 enabling recreation of the first execution environment based on the first repository version number and the second execution environment based on the second repository version number at a later time.

20. The machine-storage medium of claim 17, wherein the first package dependency and the second package dependency are associated with different computing architectures,

27

the first execution environment configured to support a first computing architecture and the second execution environment configured to support a second computing architecture.

21. The machine-storage medium of claim 17, the operations further comprising:

managing, via the database system, the repository of the packages, wherein the managing further comprises:

logging the generating of the first execution environment;
logging the generating of the second execution environment;

retrieving the first package dependency and the second package dependency from the repository of the packages;

logging the retrieving of the first package dependency;
and

logging the retrieving of the second package dependency.

22. The machine-storage medium of claim 17, wherein generating the results data further comprises:

28

aggregating a first output from the first execution environment; and

aggregating a second output from the second execution environment.

23. The machine-storage medium of claim 22, wherein generating the results data further comprises:

passing the first output from the first execution environment as input to the second execution environment.

24. The machine-storage medium of claim 22, the operations further comprising:

generating the first output based on execution of a user-defined function in the first execution environment; and

generating the second output based on the execution of the user-defined function in the second execution environment.

* * * * *